2.3 发送第一条消息

> **生成模型**：OpenAI GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~25k tokens，输出 ~3.2k tokens（本节）

前两节做的所有准备，都是为了这一刻：让 OpenClaw 真正回你一句话。

如果你是第一次搭这种系统，看到模型回复从自己的终端里冒出来，那个感觉会很不一样。因为从这一刻开始，它不再是“我读过介绍、知道它大概能干嘛”的项目，而是“我机器上已经跑起来的一个系统”。这一步非常关键。

这一节我们做三件事：

1. 启动 Gateway
2. 通过 CLI 发出第一条消息
3. 用最简化的方式理解背后到底发生了什么

不用担心，我们这里先不碰复杂通道，也不分析深层源码。先把手感建立起来。

2.3.1 先启动 Gateway

Gateway 是 OpenClaw 的中枢进程。你可以把它想成一个本地总控台：CLI、通道、模型请求、会话状态，都会经过它。没有它， `openclaw agent` 这类命令就没有地方可连。

先在一个终端窗口里运行：

```
openclaw gateway
```

如果你想第一次看得更清楚一点，可以加上详细日志：

```
openclaw gateway --verbose
```

正常启动时，你看到的日志可能类似这样：

```
[gateway] Starting OpenClaw Gateway...
[gateway] Loading configuration from ~/.openclaw/
[gateway] Listening on 127.0.0.1:18789
[gateway] Ready
```

不同版本的输出会不一样，但有两个关键词你可以留意：

- `Listening on ...`
- `Ready`

只要看到这类信息，通常就说明 Gateway 已经在本机监听了。

注意，运行这个命令的终端先别关。Gateway 前台运行时，关掉窗口它就停了。你可以把这个窗口理解成“服务窗口”，接下来再开第二个终端做消息测试。

2.3.2 启动后先做一个小检查

在第二个终端里，先别急着问复杂问题。可以先查一下 Gateway 状态：

```
openclaw gateway status
```

预期输出大概会像：

```
Gateway status: online
Host: 127.0.0.1
Port: 18789
```

如果这里提示离线，那就说明问题还在 Gateway 启动阶段，先回到上一个终端看日志，不要继续往后跑。

2.3.3 发出第一条消息

现在来最关键的一步。在第二个终端里执行：

```
openclaw agent --message "你好，介绍一下你自己"
```

如果你的模型和 Gateway 都配置好了，终端里很快就会开始出现回复。示意输出可能像这样：

```
OpenClaw:
你好，我是运行在你本机上的 OpenClaw 助手。你可以把我理解成一个通过 Gateway 管理会话、模型和通道的 AI
```

这句回复的内容不一定跟书里一模一样，毕竟不同模型的表达会不同。但只要你看到了自然语言回复，就说明最关键的第一步成了。

如果你想再试一条更实用的

比如：

```
openclaw agent --message "请用三句话解释 OpenClaw 和普通聊天机器人有什么不同"
```

你可能看到类似输出：

```
OpenClaw 更像一个运行在你自己设备上的 AI 助手底座，而不是单纯的网页聊天框。
它有 Gateway、会话、模型配置、通道接入和工具系统这些运行时能力。
普通聊天机器人更偏单一入口，OpenClaw 则强调把不同入口统一到同一套本地控制平面里。
```

如果这条也能正常回来，说明你的环境已经不是“勉强装上”，而是真的能用了。

2.3.4 如果没有回复，先看哪儿

第一次跑不通太正常了，别怀疑人生。按下面这个顺序排最省事。

第一步：看 Gateway 终端有没有报错

你前面启动 `openclaw gateway` 的那个窗口，是最直接的信息源。常见问题有：

- 端口冲突
- 配置文件损坏
- 模型认证信息不可用
- 启动后立刻退出

如果那里已经在报错，先修它，不要只盯着 `agent` 这边。

第二步：跑 `openclaw doctor`

```
openclaw doctor
```

它会帮你快速检查环境、配置、Gateway 可达性等。很多时候你以为是“模型没回”，实际上是“Gateway 根本没起来”或者“默认模型没有配成功”。

第三步：确认默认模型真的配好了

如果你在引导向导里填了 API Key，但没选好默认模型，或者选了一个当前账号不能调用的模型，也会表现成“我发出去了，但没得到像样的回复”。

第四步：确认网络环境

OpenClaw 在本地跑，但模型调用大多还是要出网。你的终端所在环境如果访问不到模型提供方，第一条消息当然回不来。Windows + WSL 用户尤其要记得检查这一层。

2.3.5 前台模式和后台模式，先别搞混

这一节我们用的是前台模式，也就是直接在终端里跑：

```
openclaw gateway
```

它的好处是最直观。日志直接在眼前，出了错也方便看。

等你以后想让 Gateway 长期驻留、开机自启，再考虑守护进程模式。那是部署和日常使用的话题，不是本章的重点。对入门来说，前台启动反而更好，因为你能亲眼看到系统状态。

2.3.6 这条消息背后到底发生了什么

现在我们来做一点非常重要、但仍然保持简单的理解。

你刚才执行了：

```
openclaw agent --message "你好，介绍一下你自己"
```

这件事并不是“CLI 自己直接去调用模型”。更准确地说，背后的流程是这样的：

```
你在终端输入消息
       ↓
`openclaw agent` 把请求发给本地 Gateway
       ↓
Gateway 读取配置，确定默认模型和当前会话
       ↓
Gateway 把请求转给对应的模型提供方
       ↓
模型返回回复内容
       ↓
Gateway 把结果交回 CLI
       ↓
CLI 在终端里显示出来
```

这条链路非常重要，你最好真的记住。因为后面不管你是从 CLI 发消息，还是从 Telegram、Discord、Slack 发消息，中间负责协调的那一层通常还是 Gateway。入口可以换，大脑和控制平面不一定换。

为什么非要多一层 Gateway

这个问题问得很好。因为如果没有 Gateway，很多能力就很难统一：

- 会话历史谁来管
- 默认模型和路由规则谁来管
- 多个消息通道怎么接到同一套运行时上
- 后面工具、技能、节点这些能力往哪儿挂

所以你可以把 Gateway 理解成 OpenClaw 的“总调度员”。CLI 只是一个前门，不是整个房子。

2.3.7 你现在已经完成了一个最小闭环

到这里，其实你已经完成了 OpenClaw 的最小可运行闭环：

```
安装 CLI / 构建源码
    ↓
运行 onboard 完成首次配置
    ↓
启动 Gateway
    ↓
用 openclaw agent 发消息
    ↓
收到模型回复
```

这条链路一旦通了，后面很多内容你学起来会轻松很多。因为你不再是对着一堆名词发呆，而是已经有了一个活的参照物：哦，原来 Chapter 3 里讲的 Gateway，就是我刚才亲手启动的那个进程；Chapter 25 里讲的 `agent` 命令，就是我刚才发消息那个入口。

这就是为什么我一直觉得，实操型入门章节很重要。它不只是教你敲命令，更是在你脑子里先搭一张地图。

2.3.8 建议你现在顺手再做两个小实验

如果你已经成功收到了第一条回复，可以再试两个很短的实验，加深手感。

实验一：改一个问题，看看回复是不是稳定

```
openclaw agent --message "请列出 3 个 OpenClaw 的典型使用场景"
```

如果能稳定返回，说明你的模型访问不是偶然撞上的一次成功。

实验二：停掉 Gateway，再发一次

先在 Gateway 窗口按 `Ctrl+C` 停掉服务，然后再执行：

```
openclaw agent --message "你还在吗？"
```

这时如果命令失败，反而是件好事。因为它能帮你直观建立一个认知： `agent` 不是独立工作的，它依赖 Gateway。这比你只在纸面上记一句“Gateway 是中枢”更牢。

2.3.9 学到这里，下一步看什么最合适

如果你此刻已经能稳定完成“启动 Gateway + 发消息 + 收回复”，那你其实已经具备了继续读后面章节的基础。接下来最自然的方向有两个：

- 往里走：去看 Gateway 架构、协议、会话、Agent 运行时
- 往外走：去接 Telegram、Discord 这类外部通道

这本书后面会两条线都覆盖，但顺序上会先带你把 OpenClaw 的内部骨架看清楚。因为只有内部图景先立住了，你再看多通道、工具、技能、记忆系统，才不会散。

本节小结

- 启动 `openclaw gateway` 之后，不要关掉那个终端；它就是本地中枢进程。
- 用 `openclaw agent --message "..."` 发出第一条消息，是验证 OpenClaw 是否真正可用的关键动作。
- 看到自然语言回复，说明 CLI → Gateway → 模型 → Gateway → CLI 这条最小链路已经打通。
- 如果没回复，优先看 Gateway 日志、 `openclaw doctor` 、默认模型配置和网络环境，不要盲目重装。
- 你刚完成的不是一次普通问答，而是 OpenClaw 整个运行时第一次在你的机器上真正工作起来。